

面向边缘 AI 大语言模型推理的 RISC-V 自定义 vdot 指令设计

——基于“香山”昆明湖 V2 处理器架构

2026 CIE 全国 RISC-V 高水平创新及应用大赛（应用方向）

August 15, 2026

目录

- 1 需求分析与方案总览
- 2 指令集设计 (ISA)
- 3 微架构设计
- 4 软件栈与工具链
- 5 验证与评估方案
- 6 实施计划与当前进度
- 7 软硬件环境与 FPGA 平台
- 8 总结与展望

背景：点积是 LLM 推理的性能与能效瓶颈

点积 (Dot Product) 是深度学习最核心的计算内核

- Transformer LLM: 自注意力 QK^T 与 AV 矩阵乘、FFN 两次 GEMM 均由海量点积构成;
- 以 GPT-2 为例, 点积占总计算量 70% 以上;
- 推荐系统中, 78% 的个性化推荐以点积作为核心排序指标;
- 边缘端部署时, 点积执行效率直接决定推理延迟、吞吐与能效。

为什么是 INT8 量化 (W8A8)

- 能效: INT8 乘法器面积/功耗约为 FP32 的 $1/10 \sim 1/20$, 算力密度 (TOPS/mm²) 远高;
- 精度: PTQ (per-tensor / per-channel / per-group) 在 7B 以下模型可保持接近 FP16 精度;
- 生态: llama.cpp / ggml、vLLM、TensorRT-LLM 均有成熟 INT8 量化与 GEMM kernel。

结论: 一条高效的 INT8 点积指令是边缘 LLM 推理的核心加速原语。

缺口：RVV 1.0 没有原生整数点积指令

昆明湖 V2 (Kunminghu V2) 完整实现 RVV 1.0

- VLEN=128, SEW=8/16/32/64, 支持掩码、尾元素、分段访存;
- 但 RVV 1.0 没有一条“一组 INT8 元素点积累加”的原生指令。

现有做法的代价

- 组合 ‘vzext ‘ / ‘vsext ‘ + ‘vwmaccu ‘ + 位宽加宽 + ‘vnsrl ‘ 打包完成 INT8→INT32 累加, 需多条指令;
- 指令数多、寄存器压力大、动态功耗高、吞吐低——K 维累加被“拆包 + 加宽 + 打包”胶水指令拖累。

行业对照：生态正走向“原生 vdot”

来源	形态
RISC-V 官方 fast-track: Zvdot4a8i / Zvqdotq	4×INT8 → INT32 累加, LLVM 已有支持 PR
SiFive: Xsfvdot / Xsfvq-macc	自定义 INT8 矩阵乘-点积, QEMU/LLVM 已实现
SpacemiT: XSMTVDot	自定义向量点积, 已合入 LLVM
香山昆明湖	无——本赛题切入点

结论：为昆明湖 V2 设计并实现一条自定义 vdot 指令，既解决边缘 LLM 推理实际痛点，又与官方 Zvdot4a8i 方向一致，具备前瞻性与兼容路径。

目标与验收指标

四大目标

- 1 设计指令： $4 \times \text{INT8} \times \text{INT8} \rightarrow \text{INT32}$ 累加，遵循 RVV 1.0 语义，编码尽量对齐官方 `Zvdot4a8i`；
- 2 硬件集成：解码接入 + 专用执行单元，极小面积换取数倍点积吞吐；
- 3 软件栈：NEMU/Spike golden、LLVM/binutils、INT8 量化与 GEMM/注意力 kernel、端侧小模型推理演示；
- 4 验证评估：指令级测试 + difftest + 基准（指令数/周期/吞吐/能效），给出可量化收益。

指标（验收口径，相对 RVV `vwmacc` 基线）

指标	目标值
指令数（INT8 GEMM 每输出元素）	下降 $\geq 50\%$
vdot 单元吞吐	1 条/周期（ ≥ 4 组 INT32）
新增面积（含解码/调度）	$\leq$ 全核 0.5%
能效（INT8 GEMM TOPS/W）	提升 $\geq 2\times$
difftest 回归	全量通过，0 失配
端侧小模型（ $\leq 2\text{B}$ ）INT8 推理	可运行，per-token 延迟

目标工作负载：LLM 自注意力 QK^T/AV 、FFN 两层 GEMM（ $K=4096$ ）、Embedding/输出层、推荐双塔打分——vdot 每 4 元素一次点积，一条指令完成 4 组累加。

总体技术路线（三阶段）

阶段一：环境部署与验证（基线）

xs-env + kunminghu-v2 + NEMU + emu/difftest
riscv-tests / coremark 基线数据

阶段二：vdot 设计与实现

ISA 规格 → NEMU/Spike golden → RTL
(解码 + 执行单元) → 工具链 → 单元测试

阶段三：协同仿真与评估

difftest 回归 → 基准 (micro/GEMM/端到端)
→ 面积功耗时序评估 → 文档与提交

关键决策点

- 1 指令形态：RVV 1.0 编码空间的 $4 \times \text{INT8} \rightarrow \text{INT32}$ 点积（对齐 Zvdot4a8i 语义），继承 vl/vstart/掩码/尾元素语义；
- 2 硬件实现：独立小执行单元挂入向量 ExeBlock，复用 VPRF 读/写回端口；备选扩展 VectorMul 数据通路；
- 3 软件顺序：golden model (NEMU) 先行 → 内联汇编/intrinsic 先行验证算法 → LLVM 完整支持后置；
- 4 验证策略：golden-model 差分随机测试为主，difftest 全量回归兜底。

四大创新点

① 指令级创新

面向 INT8 量化 LLM 推理的 $4 \times \text{INT8} \rightarrow \text{INT32}$ 原生点积指令，语义完整对齐 RVV 1.0 (vl/vstart/掩码/尾元素/异常)，编码与官方 fast-track 提案 Zvdot4a8i 一致，具备直接演进为官方扩展的路径。

② 微架构创新

以“极小面积开销换取数倍点积吞吐”为目标，给出乘加阵列 + 加法树 + 累加器数据通路设计、时延/吞吐权衡与面积功耗估算方法。

③ 系统级创新

打通“ISA $\rightarrow$ 硬件 $\rightarrow$ golden model $\rightarrow$ 编译器 $\rightarrow$ 量化 $\rightarrow$ GEMM kernel $\rightarrow$ LLM 推理”全链路，交付可复现的端到端演示与评估数据，而非仅有 RTL 代码。

④ 工程创新

利用香山敏捷开发基础设施 (difftest、随机差分测试、emu 周期仿真) 建立“指令语义 $\rightarrow$ 硬件 $\rightarrow$ 参考模型”三方一致的可验证闭环。

设计原则

- 1 RVV 1.0 语义一致：完整继承
‘vl’ / ‘vstart’ / 掩码/尾元素/异常语义，
不发明新执行模型——与 ‘vsetvl’ 无缝协作、golden 语义唯一、编译器可安全处理；
- 2 与官方方向对齐：语义与助记符完全对齐
Zvdot4a8i，作为自定义扩展 ‘Xkhmvdot’
先行实现，官方发布后仅改扩展名即可平滑迁移；
- 3 最小侵入：编码落在 RVV OP-V 主操作码
(‘1010111’) 未占用空间，不触碰已实现
RVV 1.0 编码；
- 4 可验证：每条指令有可执行伪代码与参考实现，
支撑差分随机测试。

扩展命名与使能

项目	值
扩展名	Xkhmvdot
助记符	vdota4/vdota4u/ vdota4su/vdota4us
使能	v1.0 恒使能（无独立 CSR）
对应官方	Zvdot4a8i（Ratification 中）

8 条指令 = 4 种符号性 × 2 种操作数形式 (vv / vx)。

指令形态总览 (8 条)

指令	操作数	vs2 元素	vs1 元素	vd 累加	语义
<code>vdota4.vv</code>	<code>vv</code>	<code>int8×4</code>	<code>int8×4</code>	<code>int32</code>	有符号 × 有符号
<code>vdota4u.vv</code>	<code>vv</code>	<code>uint8×4</code>	<code>uint8×4</code>	<code>uint32</code>	无符号 × 无符号
<code>vdota4su.vv</code>	<code>vv</code>	<code>int8×4</code>	<code>uint8×4</code>	<code>int32</code>	有符号 × 无符号
<code>vdota4us.vv</code>	<code>vv</code>	<code>uint8×4</code>	<code>int8×4</code>	<code>int32</code>	无符号 × 有符号
<code>vdota4.vx</code>	<code>vx</code>	<code>int8×4</code>	<code>rs1 低 32 位</code>	<code>int32</code>	标量广播 (4× <code>int8</code>)
<code>vdota4u.vx</code>	<code>vx</code>	<code>uint8×4</code>	<code>rs1 低 32 位</code>	<code>uint32</code>	标量广播
<code>vdota4su.vx</code>	<code>vx</code>	<code>int8×4</code>	<code>rs1 低 32 位</code>	<code>int32</code>	标量广播
<code>vdota4us.vx</code>	<code>vx</code>	<code>uint8×4</code>	<code>rs1 低 32 位</code>	<code>int32</code>	标量广播

- 数据视图 (关键约定): 所有向量操作数以 `SEW=32` 元素视图访问 (‘`vsew`’ 必须为 32); 每个 32 位元素内部重新解释为 4 个打包的 8 位子元素 (little-endian, 子元素 `j` 位于位 `[8j, 8j+7]`)。
- 元素计数 (`vl/vstart`) 单位是 32 位元素, 无 `vl%4` 对齐约束。
- `.vx` 形式将 `rs1` 视为 “打包的 4 个 8 位元素”, 用于权重/常量广播场景 (如 QK^T 一侧固定向量、推荐打分)。

语义精确定义（伪代码）

‘vdota4.vv vd, vs2, vs1 ‘ 伪代码（权威定义）

```
SEW = 32
VLMAX = LMUL * VLEN / SEW
for i in [vstart, vl):
    if vm == 0 and mask[i] == 0:
        continue # 该元素不更新
    acc = SExt32(vd[i]) # 读 vd 旧值
    for j in 0..3:
        a = Sub8(vs2[i], j) # 位 [8j, 8j+7]
        b = Sub8(vs1[i], j) # 位 [8j, 8j+7]
        acc = acc + a * b # 乘积 16 位, 累加 32 位
    vd[i] = acc
# 尾元素按 vta 策略处理
# vstart≠0 → illegal instruction (与昆明一致)
# 所有向量算术指令一致)
```

符号性矩阵

变体	vs2	vs1	累加器
vdota4	s8	s8	s32 回绕
vdota4u	u8	u8	u32 回绕
vdota4su	s8	u8	s32 回绕
vdota4us	u8	s8	s32 回绕

溢出与饱和策略

- 指令内：单次点积范围 $\subseteq [-65536, +64516]$ ，16 位有符号可完整表示，不溢出；
- 跨指令累加：32 位模 2^{32} 回绕 (wrapping)，与 RVV ‘vadd.vv ‘ 一致，不做饱和——INT8 GEMM 的 K 维累加 ($K=4096 \rightarrow \approx 2^{28}$) 在 int32 内天然安全。

vstart / 掩码 / 尾元素与 RVV 交互约束

三种机制

机制	定义
vstart	v1.0 与昆明湖一致: $vstart \neq 0$ 的向量算术指令为 legal instruction (RVV 规范允许仅支持 vstart=0) 'vsetvl' 清零, vdot 无需新增机制
掩码 vm	vm=0 时读掩码寄存器 (EEW=1), 掩码位 i 对应元素 i 掩码为 0 的元素不读源、不计算、不写 vd
尾元素 vta	v1.0 按 undisturbed 处理 (写回旧值); vta=0 时 执行单元内 'Mgu' 统一处理

与 RVV 1.0 交互约束 (违反即 illegal)

#	约束
C1	'vsew' 必须为 32
C2	$VLMAX \geq 1$ 且寄存器组不越界 ($LMUL \in \{1, 2, 4, 8\}$)
C3	向量扩展已使能 ('mstatus.VS' $\neq$ Off)
C4	编码落入已定义空间但组合未定义 $\rightarrow$ illegal

除上述外: 不产生任何访存、无副作用
CSR 更新、无特权级限制, 任何模式下
可执行 (与 RVV 算术指令一致)。

编码 (RVV OP-V 空间)

编码模板: OP-V 主操作码 '1010111' (0x57)

funct6 [31:26]	vm [25]	vs2 [24:20]	vs
----------------	---------	-------------	----

- funct3 选型: 乘累加/宽乘累加族
('vmacc ' / 'vwmaccc ' / 'vwmacccu
统一使用 010 (vv) / 110 (vx); vdot
语义上属“打包 4 元素宽乘累加”, 故采用
同类别, 与官方 Zvdot4a8i 方向一致;
- vm=0 掩码使能; vv 形式 vs1 为向量, vx
形式为 rs1。

v1.0 编码定稿 (已与实现同步)

指令	funct6	funct3	状态
vdota4.vv	111001	010	✓ 已实现
vdota4.vx	111001	110	□ 预留
vdota4u.vv	010100	010	□ 预留
vdota4u.vx	010100	110	□ 预留
vdota4su.vv	010101	010	□ 预留
vdota4su.vx	010101	110	□ 预留
vdota4us.vv	010110	010	□ 预留
vdota4us.vx	010110	110	□ 预留

编码核对已完成: 111001 经脚本扫描
rocket-chip 全部 OP-V BitPat + 香山自
建表, 无冲突 (与 VWMACCU_VV 同格
式); 剩余核对项 (官方定稿比对、反汇编
round-trip) 待官方 Zvdot4a8i 发布/工
具链落地时执行。

指令行为汇总（验证测试矩阵依据）

场景	期望行为
正常 vv , $vl=VLMAX$	全量计算, 结果正确
$vl=0$	无操作, vd 不变
掩码全 0 / 部分 1	vd 不变 / 仅掩码位 =1 元素更新
$vstart > 0$	illegal instruction (与昆明湖所有向量算术指令一致)
尾元素 ($i \in [vl, VLMAX)$)	按 vta 策略 (undisturbed/agnostic)
符号混合变体 (su/us)	按符号矩阵计算
溢出回绕	模 2^{32} 回绕 (可构造边界用例)
$SEW \neq 32$ (8/16/64)、 $VS=Off$ 、未定义编码	illegal instruction
vx 形式	$rs1$ 低位 32 位作为打包 4x8bit 广播

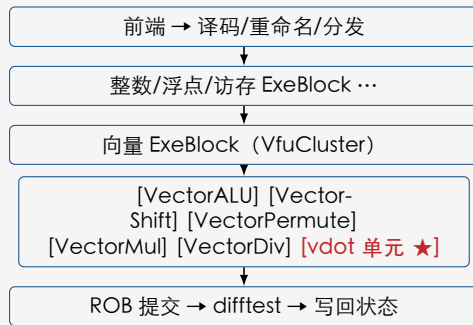
演进路径: $v1.0$ (本赛题交付, 8 条指令恒使能) $\rightarrow v1.1$ (扩展使能 CSR, 虚拟化/多核选择性使能) $\rightarrow v2.0$ (官方 $Zvdot4a8i$ 发布后切换官方编码与扩展名; 增加饱和变体、 $INT8 \times INT8 \rightarrow INT64$ 长累加对齐 $Zvqdotq$)。

微架构设计目标与总体思路

目标	约束	策略
极小面积开销	$\leq$ 全核 0.5%	8×8 乘法阵列 + 加法树
数倍吞吐	1 条/周期	乘加阵列流水化, 独立
最小侵入	不动前端/访存/ROB	仅改解码表 + FuType+
RVV 语义完整	vl/vstart/掩码/尾/异常	复用向量后端既有机制

- 源码基线: kunminghu-v2 分支 commit e12436c7 (2026-08-14);
- 结构提示: 向量执行单元实际在 backend/fu/wrapper/ (VAluFix/VIMacU/VIDiv/VIPU/VPPU/...), 公共框架在 backend/fu/vector/ (VecPipedFuncUnit/Mgu/...);
- 昆明湖没有任何现成自定义指令钩子 (无 xvld/xvst/XCustom), vdot 必须走完整标准链路。

总体位置 (示意)



共享 VPRF 读/写端口、writeback 端口

解码接入（遵循昆明湖既有 RVV 链路）

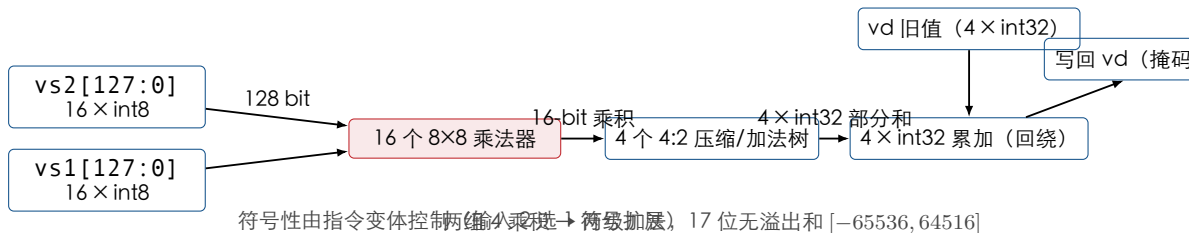
以 ‘vwmaccu.vv’ 为样板，每环均已定位：

- 1 编码 BitPat: Instructions.scala 登记 VDOT_VV = BitPat("b111001?????????010?????1010111") (v1.0 已实现)；
- 2 解码表: VecDecoder.scala 加入 VDOTA4_VV -> OPMVV(T, FuType.vdot, VdotType.vdota4, F, T, F, UopSplitType.VEC_VVV)——每 128-bit 输出 4×32bit 结果，不真正加宽，复用 VEC_VVV (1 uop/lmul)；
- 3 fuOpType: 新增 VdotType (9-bit OpType: vs2/vs1/vd 符号 + format + 子操作码)；
- 4 FuType: 新增 vdot，同步加入 vecOPI/vecArith/vecAll 集合——漏改任一集合会导致 dispatch 找不到可接受 EXU；
- 5 illegal 检查: VecExceptionGen.scala 补充 vdot 的 EEW/EMUL/重叠检查（数据视图特殊，不直接落入 ‘vdWideningInst’）；vstart≠0 illegal 与执行侧兜底自动覆盖；
- 6 调度/写回: VdotCfg 挂入 VFEX0（复用 3×VfRD + VORD + VIRL 与 VfWB(0,0)）——不新增 VPRF 端口号。

改动预估

硬件侧约 600–1200 行 Scala (Instructions +8 BitPat; VecDecoder +8 映射; VdotType + 30 行; FuType/FuConfig + 30 行; VdotU.scala 新文件 300–600 行; Vdot64b 200–400 行; VecExceptionGen +10–40 行)。

执行单元数据通路 (VLEN=128, LMUL=1)



- 多周期流水: 建议 latency = 2 (与 VIMacCfg/CertainLatency(2)、yunsuan VIMac64b 两级寄存器结构同构; 吞吐 1 条/周期, 无结构冲突);
- 掩码/尾/vstart: 由执行单元内 Mgu (mask/tail merge unit) 统一处理——vdot 直接复用, 无新逻辑;
- LMUL: 1/2/4/8 均可寻址 (int32 元素视图, 无加宽比限制); LMUL>1 按寄存器组分段执行 (每周期一个 128-bit 片段), 吞吐相应降低。

性能建模与面积/功耗/时序估算

吞吐与数据率（设计预估）

参数	值
VLEN / 数据通路宽度	128 bit
每周期点积组数	4 (16 对 8×8 乘加)
吞吐	1 条/周期 (16 MAC/周期)
时延	2 周期 (建议)
uop 拆分	VEC_VVV (1 uop/lmul)
面积 (估算)	乘法器 3.2k + 加法树/累加 2k 门

相对 RVV 1.0 基线的收益（预估）

指标	RVV 组合	vdota4	提升
指令数/4×4	8-12	1	≥ 8×
动态功耗	高	低	显著
寄存器压力	高	低	减少 50%

面积 / 功耗 / 时序估算方法

- 面积：vdot 单元独立综合 (Synopsys DC 或开源 OpenROAD)，报告面积 ÷ 全核估计面积，目标 $\leq 0.5\%$ ；
- 功耗：综合后功耗报告（基准波形回标 switching activity）→ TOPS/W 对比；
- 时序：8×8 乘法 (0.5-1ns@28nm) + 加法树 + 32 位累加，目标频率按昆明湖主频 (1-2GHz 假设，注明口径)；
- 口径：所有数字注明工艺、库、温度/电压条件，保证可复现可审计。

备选实现方案

方案	描述
A (默认)	独立 VdotU + yunsuan Vdot64b, 2 拍, 挂 VF
B	扩展 VIMac64b 加积点模式
C	VIAIFix 多周期微码

改动点清单（代码级，kunminghu-v2 @ e12436c）

#	文件/模块（相对 src/main/scala/）	改动	估计量
1	backend/decode/Instructions.scala	新增 object Xkhmvdot (8 条 BitPat)	+8 条目
2	backend/decode/VecDecoder.scala	新增 8 条 OPMVV/OPMVX 映射 (VEC_VVV)	+8 映射
3	yunsuan package.scala + Opcode	新增 VdotType (9-bit OpType)	+ 30 行
4	backend/fu/FuType.scala	新增 vdot, 同步集合	+ 8 行
5	backend/fu/FuConfig.scala	新增 VdotCfg (仿 VimagCfg) + 谓词	+ 20 行
6	Parameters.scala (vfSchdParams:449)	VdotCfg 加入 VFEX0	+1 行
7	backend/fu/wrapper/VdotU.scala (新建)	VecPipedFuncUnit + 2×64-bit 核心 + Mgu	300-600 行
8	yunsuan vectorIMAC/Vdot64b.scala (新建)	8×8 乘法 + 加法树 + int32 累加	200-400 行
9	backend/decode/VecExceptionGen.scala	vdot 的 EEW/EMUL/重叠检查	+10-40 行
10	DecodeUnitComp.scala / UopInfoGen	复用 VEC_VVV, 无需改动	0
11	测试	Vdot64bSpec + decode 单测 + 回归	中
12	NEMU (外部仓库)	8 条指令语义 + 编码注册	中
13	difftest	无需改动 (DiffInstrCommit 已覆盖)	0

合计：硬件侧约 600-1200 行 Scala + NEMU 实现与测试。VPRF 复用现有端口（6R/4W，自动推导），面积零增长。

软件栈总览与推进顺序

软件栈（六层）

应用层：LLM 推理 (llama.cpp/自研) · 推荐打分

Kernel 层：INT8 GEMM/GEMV · QK^T /AV · 量化

Intrinsic: `__riscv_vdota4_*` / 内联汇编 / C fallback

编译器：LLVM (Xkhmvdot subtarget + intrinsic + M)

模拟层：NEMU (difftest golden) · Spike · 软件

硬件层：昆明湖 V2 RTL (vdot 执行单元)

推进顺序原则

模拟层 (golden model) 先行 → intrinsic/内联汇编让算法组先写 kernel → 编译器支持后置 → 最后集成 LLM 推理。

golden model

- NEMU: 按 02 §4.2 伪代码实现 8 条指令语义；必须在解码表/扩展注册处登记自定义编码（否则报 illegal 与硬件失配）；差异点清单随代码交付；
- Spike: `riscv/insns/vdota4*.h` + 解码表 + 扩展名注册
(`-march=rv64gcv_xkhmvdot`)；
- 软件仿真器（开发期 fallback）：逐元素仿真，作为“第 3 个参考实现”参与随机差分

量化方案与 INT8 GEMM kernel

量化策略分级

级别	粒度	精度	vdot 适配
L1	per-tensor	一般	直接支持
L2	per-channel	好	权重打包带 scale 向量
L3	per-group	最好	反量化循环按 group 取 scale

- v1.0 交付 L1 + L2, L3 作扩展; 对称 INT8 为默认;
- 反量化移出 K 循环 (K 维累加保持 int32, vdot 只承担最重乘加);
- 权重打包: 每个 32 位字装 K 维方向相邻 4 个 int8 (离线静态完成, 运行时零开销); K 非 4 倍数 padding 补 0。

INT8 GEMM/GEMV microkernel (核心交付)

```
// M=1 GEMV, K=4096, N 分块
for (n = 0; n < N; n += 4) {           // 4 个
    vint32m1_t acc = 0;
    for (k = 0; k < K; k += 4) {
        vint32m1_t xv = vle32(&x_packed[k/4])
        vint32m1_t wv = vle32(&w_packed[k/4])
        acc = __riscv_vdota4_vv_i32m1(acc, xv, wv)
    } // 一条指令 4 组点积
    // 反量化 + 写回
}
```

收益 (理论): 每 4×4 乘加由 8-12 条指令降为 1 条 vdot + 2 条 vle32, 指令数下降 ≥ 60%, 寄存器压力减半。

注意力 kernel 与 LLM 推理集成

注意力 kernel

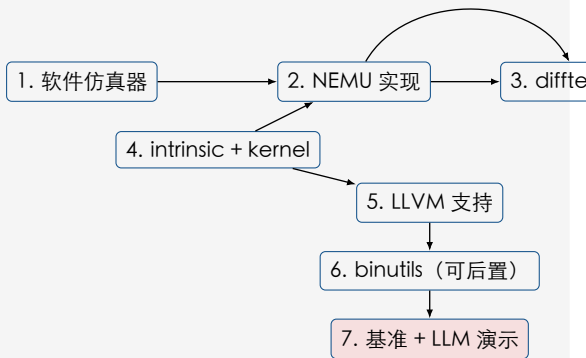
- QK^T : $M=1$ (单 token), $K=hdim$ (如 128/group), vdot 逐组点积 $\rightarrow$ fp32 $\rightarrow$ softmax;
- AV: $S \times V$, vdot 累加 $\rightarrow$ 输出 token;
- 长序列分段 (如 $S=4096$ 按 chunk) 控制寄存器与延迟。

LLM 推理集成 (两条路径)

路径	说明
A. 自研轻量栈	注意力 + FFN + 量化 + $\leq 2B$ 模型 INT8
B. llama.cpp/ggml	替换 q8_0 GEMM 向量化路径

建议: 路径 A 为主交付 (可控、可演示、可复现); 路径 B 作为加分项贡献上游。

工具链打通顺序 (依赖图)



★★★ 主交付

软件侧交付物

NEMU/Spike patch、软件仿真器、intrinsic 头文件、LLVM patch (目标含入

验证目标与层次

层次	对象	方法
L0 指令语义	仿真器/NEMU/Spike	指令级测试 + 随机差分
L1 硬件功能	vdot RTL	定向 + 随机 + difftest
L2 系统集成	昆明湖 V2 全核	riscv-tests+ 向量测试 +
L3 性能/能效	单元/GEMM/LLM	基准（口径见 §5）

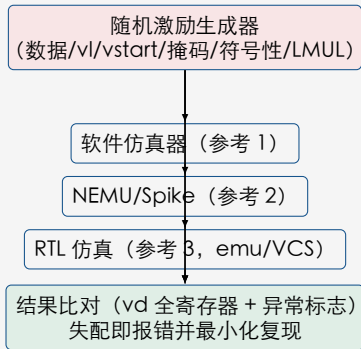
核心思想：功能验证以“三方一致 + difftest 兜底”为核心；
评估以“可复现、口径明确”为原则。

指令级测试矩阵（节选）

类别	覆盖点
基本	4 符号性 × 2 形式 × 手工算例
vl 边界	vl=0、1..VLMAX-1、VLMAX、>VLMAX
掩码	全 1/0、交替、随机
vstart	=0 正常；≠0 → illegal
尾策略	vta=0/1、尾元素旧值保持
溢出	全 127、全 -128、多轮累加
非法	SEW∈8,16,64、VS=Off、未定义编码
寄存器组	LMUL=1/2/4/8、越界检测
数据布局	打包子元素边界、大小端
vx 形式	rs1 各 4 字节组合、符号性
随机	随机 vl/vstart/掩码/数据/符号性

随机差分（三方一致）与 difftest 协同仿真

随机差分测试



- 覆盖: SEW×LMUL×vl×vstart×掩码×符号性×数据模式;
- 数量: 随机用例 ≥ 1 万条, --seed 可复现;

difftest 协同仿真

- 框架: `emu --diff riscv64-nemu-interpretor-s`
- 前提: NEMU 已实现 8 条指令, 否则必然失配;
- 测试负载: riscv-tests 全量 / 自研向量测试集 / 随机差分镜像 / kernel 冒烟 (GEMM 与 fp32 参考比对误差界) / LLM 推理冒烟 (per-token 输出比对);
- 通过标准: 全部 difftest 0 失配、0 死锁。

回归策略与 CI

- 每日回归: L0 + L1 difftest 冒烟 (固定 seed 集);

性能与能效基准（口径定义）

基准	度量	对比对象
B0 micro	吞吐、时延	vadd/vwmacc 基
B1 GEMM	指令数、周期、寄存器压力	vwmacc 组合
B2 端到端	per-token 延迟、吞吐	FP16/INT8 基线
B3 能效	TOPS/W	vwmacc 组合路径
B1 指令数理论对比 (VLEN=128, LMUL=1, K=4):		
实现	每 4×4 乘加排	
RVV 组合 (vzext→vwmaccu→加宽→vnsrl)	8-12 条	
本设计 vdot4	1 条	

实验对照原则

- 同一 kernel 框架、量化参数、分块参数下，仅替换“点积原语”，保证对比公平；
- 数据可复现：固定 seed、固定镜像、记录环境（commit SHA、编译选项、仿真器版本）。

面积/功耗/时序评估

- 面积：独立综合（DC 或 OpenROAD），占比 $\leq 0.5\%$ ；
- 功耗：波形回标 switching activity → TOPS/W；
- 时序：关键路径分析 vs 目标频率，不满足则流水切分（备选方案 B）。

实施计划：里程碑总览（三阶段 + FPGA 加分项）

阶段	目标	关键产出
阶段一环境部署与验证（2-3 人周）	打通开发与仿真基线	xs-env、emu、difftest 基线
阶段二 vdot 设计与实现（5-7 人周）	指令从规格到 RTL 落地	ISA 定稿、golden、RTL、intrinsic、指令级测试
阶段三协同仿真与评估（3-4 人周）	证明正确与收益	全量回归、B0-B3、面积报告、LLM 演示
阶段四 FPGA 原型验证（6-10 人周，加分）	单卡运行 V4-Flash 同构模型	vdot+ 香山子集核上板、INT8 GEMM 阵列

关键路径：1.2 → 1.4 → 2.3 → 2.5 → 3.1 → 3.2

主要风险与缓解

- 可并行支线：golden/kernel (2.2/2.6/2.7) 与 RTL (2.3-2.5) 互不阻塞；LLVM (2.8) 全程并行；
- 阶段二预留 1 周缓冲。
- 官方编码冻结冲突 → 以官方为准切换（改动集中解码表 + 工具链）；
- 时序不满足 → 流水切分 / 扩展 VIMac64b 复用 2 级寄存器；
- difftest 失配 → 三方随机差分先行定位、失配自动最小化；
- LLVM 工作量超预期 → 内联汇编 intrinsic 保底；
- 源码/环境问题 → xs-env 官方流程 + 大赛答疑通道。

最终交付物清单与提交

类别	交付物
文档	设计文档套件 (01-06)
RTL	解码 + 执行单元 + 后端集成 patch (含单测)
Golden	NEMU/Spike patch + 软件仿真器
工具链	LLVM patch + intrinsic 头文件 + 打包器
软件	INT8 GEMM/注意力 kernel、量化脚本、LLM 演示
验证	指令级测试、随机差分、回归脚本与报告
评估	B0-B3 报告 (性能/能效/面积)
提交	my_submission.txt、PR 描述

my_submission.txt 模板

team-name: (团队名称)
repo-url: github.com/...git
github-username: (用户名)
contact-email: (邮箱)
project-description: 面向边缘AI LLM推理的RISC-V自定义vdot指令 (4x INT8→INT32点积) 设计, 基于香山昆明湖V2实现解码、执行单元与软件栈, difftest验证+GEMM/端到端基准评估
university: (学校/单位)
advisor: (指导老师)
team-members: 成员1(架构), 成员2(硬件), 成员3(软件), 成员4(验证)
notes: 语义对齐RISC-V官方Zvdot4a8i fast-track提案, 平滑演进路径

当前进度：任务完成情况 CheckList

总览 (2026-08-15)

维度	状态	说明
源码工作副本	✓	xs-kunminghu-v2 + yunsuan
yunsuan 实现	✓	Opcode/Type/Vdot64b/测试
XiangShan 集成	● 部分	代码已接入，待整机编译
指令编码	✓	funct6=111001，脚本核对无冲突
Vdot64bSpec	✓	固定 +32 组随机用例全部通过
NEMU golden	✓	vdot.vv 已实现并实测通过
difftest 回归	□	待完整 Linux 工具链

总体进度：硬件侧核心实现约 70% (RTL 代码完成，整机编译与协同仿真未跑)；软件侧 golden 已启动。

已交付代码 (节选)

- yunsuan: VdotOpcode.scala、VdotType、Vdot64b.scala、Vdot64bSpec.scala (chisel 6.7.0 全量编译通过)；
- XiangShan: VdotU.scala (新)、FuType/FuConfig/Instruction 6 处修改；
- NEMU: vdot_instr + 解码注册 + vdot_test.S——实测 lane0=0x0a，两次累加 =0x14。

待办 (P0-P3)

P0: 完整 clone 整机编译；P1: decode 单测 + riscv tests 随机差分, P2: 应用方向

结论速览

待办	建议环境
P0-1 NEMU 实现	✓ 已完成 (macOS 本机)
P0-2 整机编译	Linux $\geq$ 16GB (推荐 32GB)
P1 回归/随机差分	同 P0-2
P2 LLVM/kernel	Linux (8 核 +)
P3 B2 LLM 演示	Linux + GPU (可选)
P3 B3 综合评估	EDA 工作站 / OpenROAD+PDK

- 本机 (macOS M2/8GB) 定位: 代码编写、yunsuan 单测、文档与 ISA 核对 (已验证可跑通);
- 全核 make emu、difftest、B2/B3 不建议本机运行。

本机差距清单 (节选)

需求项	现状	获取方式
Linux 环境	macOS	云主机/Docker
mill	×	Maven 脚本安装
RISC-V gcc	×	apt / toolchain
完整 XiangShan	tarball	clone -recursive
内存 $\geq$ 16GB	8GB	云主机 32GB
EDA + PDK	×	OpenROAD + Sky130/ASAP7

分阶段落地

阶段 A (本机): 代码/单测/文档 → 阶段 B (云主机 8c/32GB/200GB): xs-env + clone + emu + difftest 回归 → 阶段 C: OpenROAD 开源综合流程。

FPGA 加速平台：单卡运行 V4-Flash 同构模型（11）

目标与定位

- 主交付：单卡运行“V4-Flash 同架构小型化模型”（DeepSeekMoE+ 混合注意力同构，20B 总参/2B 激活，INT8）， ≥ 50 token/s；
- 扩展交付：4-8 卡运行全量 284B（INT2 71GB / INT4 142GB）；
- FPGA 平台是 vdot 指令的系统级验证与演示载体：vdot 单元在 PL 实现作为 GEMM 加速原语，香山核做控制/调度/专家路由。

可行性先算账（全量 284B）

约束	需求	结论
容量	INT4=142GB / INT2=71GB	单卡装不下，需 4-8 卡
算力	50 t/s = 1.3 TMAC/s	1 张 Versal 即接近（不是并
带宽	325 GB/s（摊 8 卡 40GB/s）	8 卡 HBM 819GB/s/卡有余

卡型选型（AMD/Xilinx）

卡型	内存	带宽	结论
VHK158	32GB HBM2e	819GB/s	★ 主选
VCK5000	32GB HBM2e	819GB/s	备选（AIE）
U55C	16GB HBM2e	460GB/s	预算降档
U250	64GB DDR4	77GB/s	× 带宽不足

单卡性能预估

模型 20B/2B INT8 @32K ctx：算力需求 0.2 TMAC/s、带宽 100 GB/s；VHK158 提供 1.4 TMAC/s / 819 GB/s（利用率 14%/12%）→ 预估 50-100+ token/s。

云上等价目标（2026-08）

AWS F2（AMD VU47P，16GB HBM + 64GB DDR4）：f2.xlarge 单 FPGA \$1.98/mh
2024 Q4 RISC-V 大算力应用方向

片上计算架构与多卡扩展

片上架构（单卡，两级算力）

PS (ARM): 驱动/推理栈 runtime、量

PL (可编程逻辑)

① 香山核 (MinimalConfig 子集, 50-100MHz): 控制平面、

② INT8 点积加速阵列 (主算力): vdot 数据视图、乘加簇/脉动阵列, 0.2-1 TMAC/s

③ 权重/激活缓存: BRAM/URAM 分块 + HBM 流式

④ PCIe Gen4 x16 DMA

- vdot 与阵列同构: 核内 vdot 指令与 GEMM 阵列共用同一量化/打包格式 (32-bit = 4×INT8), 避免两套格式;
- 昆明湖全核 FPGA 时序/面积困难 → MinimalConfig 子集降频, 仅承担控制/调度; 主算力由阵列承担。

多卡方案 (EP 专家并行)

配置	卡数	预估吞吐
INT4 (质量优先)	8× VHK158 (256GB)	40-80 token/s
INT2 (容量优先)	4× VHK158 (128GB)	20-50 token/s

- 并行度: EP 为主 (专家按卡分片, all-to-all) + TP + 序列并行 (1M ctx KV 分片) + MTP 投机解码 (可选);
- 互联: PCIe Gen4 x16 (32GB/s), 通信量 1-3 GB/s/卡不是硬瓶颈, 延迟用批量路由掩盖;
- 每卡算力/带宽富余: 163 GMAC/s/卡 vs PL 1.4 TMAC/s; 40GB/s/卡 vs 819GB/s。

单卡 vs 多卡决策

单卡 (20B/2B, INT8, 50-100+ t/s, 低风险, 主演示) vs 多卡 (全量 284B, INT2/4, 20-80 t/s, 挑战性加分项)。2026 CIE RISC-V 大赛·应用方向 30/33

硬件市场价格与启动投资估算 (12)

市场价格锚点 (2026 调研)

硬件	市场价	类型
Alveo U55C (16GB HBM2e)	¥82,828/张	实价锚点
Versal HBM VHK158 套件	¥118,000/套	实价锚点
U280 (8GB HBM+64GB DDR4)	≈¥45,000	估算
U250 (64GB DDR4)	≈¥20,000	估算
工作站 (单卡)	≈¥20,000	估算
Vivado ML Standard	免费	—

- U55C 与 VHK158 单卡价差约 ¥3.6 万;
- 全量 284B 需 4-8 张 HBM 卡 → 数十万 百万级。

投资档位 (含 10% 预备金)

档位	硬件	小计
T0 开发起步 (无 FPGA)	工作站/云主机	¥22,000
T1 经济单卡	1×U55C+ 工作站	¥118,611
T2 基准单卡 (推荐)	1×VHK158+ 工作站	¥181,292
T3 经济全量	4×VHK158+ 服务器	¥632,384
T4 旗舰全量	8×VHK158+ 服务器	¥1,208,173

F2 时代: 纯云路线优先 (2026-08)

租 vs 买结论反转: 开发/验证 + 主交付演示用 AWS F2 按小时租用全面优于购买——6 个月非 24/7 仅 ¥1.4 万 (购卡 10%); 全量 284B 用 f2.48xlarge 演示 200h ≈ ¥2.3 万。纯云方案总投入 ≈ ¥5-8 万, 无需购置实体 FPGA。

我们做了什么

- 识别边缘 LLM 推理核心瓶颈 (INT8 点积), 定位 RVV 1.0 指令缺口, 对齐官方 Zvdot4a8i 方向;
- 完成 Xkhmvdot 指令族设计 (8 条, $4 \times \text{INT8} \rightarrow \text{INT32}$, RVV 1.0 语义完整, 编码定稿 funct6=111001 无冲突);
- 给出微架构集成方案 (解码链路逐环定位 + 独立执行单元 + VPRF 端口零新增, 600-1200 行 Scala);
- 打通软件栈与验证体系 (golden $\rightarrow$ 工具链 $\rightarrow$ kernel $\rightarrow$ LLM 集成; 三方一致 + difftest);
- 已有落地成果: yunsuan Vdot64b 单测全过、NEMU golden 实测通过、硬件核心实现 70%;
- 规划 FPGA 单卡 (可扩展多卡) 运行 V4-Flash 同构模型 ≥ 50 token/s 的演示路线与投资方案。

下一步计划

- 1 完整 clone 整机编译 (P0) $\rightarrow$ decode 单测 + riscv-tests 回归 (P1);
- 2 随机差分 + difftest 全量 (P1);
- 3 intrinsic + LLVM 支持 + GEMM/注意力 kernel + B0/B1 基准 (P2);
- 4 B2 端到端 LLM 演示 + B3 面积功耗评估 (P3);
- 5 (加分) FPGA 单卡上板演示 ≥ 50 token/s。

核心价值

极小面积开销 $\times$ 数倍点积吞吐 $\times$ 全链路可验证——
为香山昆明湖 V2 补齐 INT8 LLM 推理的关键加速原语, 并预留平滑演进为 RISC-V 官方扩展的路径。

谢谢！

敬请各位评委老师指正

2026 CIE 全国 RISC-V 高水平创新及应用大赛（应用方向）

面向边缘 AI 大语言模型推理的 RISC-V 自定义 vdot 指令设计